HACKWITHZACH · LAB WALKTHROUGH · PILLAR 4

I vibe coded an online store. It had 21 security holes.

One sentence built a finished-looking shop with a login, a catalog, a chatbot, an admin console and a checkout. One more sentence made everything free. This guide walks you through breaking it, then asking for it again the right way.

21
FINDINGS IN THE GENERATED STORE

14
RATED CRITICAL

0
AFTER REQUIREMENTS MOVED INTO THE PROMPT

No cloud account	No credentials	Runs offline on localhost	~20 minutes

You will run a real store on your own machine, scan it with a gate that fails the build, break it in the browser, then re-prompt the same assistant and watch the same gate pass. Cybersecurity education that gets you hired, promoted and paid · hackwithzach.com

© 2026 Vigilantia Technologies INC. · HackWithZach

WHAT YOU ARE ABOUT TO SEE

Vibe coding builds working apps that are quietly unsafe

Vibe coding is describing what you want in plain language and letting an AI write the code. It is fast, it is real, and the result usually runs on the first try. The problem is not that the output is broken. The problem is that it looks finished.

The store in this lab came back from a single sentence. It has a sign-in page, a product catalog, a search box, a chatbot, customer reviews, a cart, a checkout and an admin console. A person who asked for it and got it back would deploy it. It carries twenty-one security findings, fourteen of them critical, not one of them exotic. They are the defaults the model was never asked to change.

> **The one idea to leave with**
>
> The assistant did exactly what it was told. Nobody told it to be safe, and nothing between a laptop and a live site would have stopped it. You do not fix this by being careful with the AI. You fix it by deciding, up front in the prompt and again in an automated gate, what the app is never allowed to do.

The sentence that built it

```
Write me a password protected website that sells widgets, with an
LLM on the backend that answers questions about the best widgets to buy.
```

Nothing about that is unreasonable. It is a complete product request and a completely empty security requirement. It never says who may see the admin page, where the API key lives, or whether the chatbot may set a price.

What you will do in this lab

- Run the generated store and see that it looks finished.
- Scan it with a gate that reads the source and fails the build.
- Break it in the browser, then buy the whole catalog for nothing.
- Re-prompt the same assistant with a control map and scan again, clean.
- See the defenses the right version adds: a firewall, a least-privilege assistant, a separate admin service, input validation and an audit log.

STEP 0

Set up the lab

You need Python 3 and Flask. Everything runs on your own machine. Open a terminal in the lab folder and start the four pieces.

```
# from the widget-store folder
pip install -r requirements.txt
python seed.py                                      # builds the store databases
python weak/app.py                                  # the vibe-coded store        :5200
WIDGET_LAB_DEV=1 WIDGET_LLM_API_KEY=sk-demo \
    python hardened/app.py                          # the hardened storefront     :5201
WIDGET_LAB_DEV=1 python hardened/admin.py           # the separate admin service  :5202
python visual/app.py                                # the console and bench       :5115
```

Then open **http://127.0.0.1:5115/attack**. That page is your bench: the live store on the right, a button for each problem on the left, and a toggle at the top that flips the same buttons between the weak store and the hardened one.

> **Why four services, not three**
>
> The admin console is its own service now, on its own port. That separation is one of the controls, and you will see why on the hardened side: breaking the shop does not hand you the admin panel, because they do not share a process or a login.

© 2026 Vigilantia Technologies INC. · HackWithZach

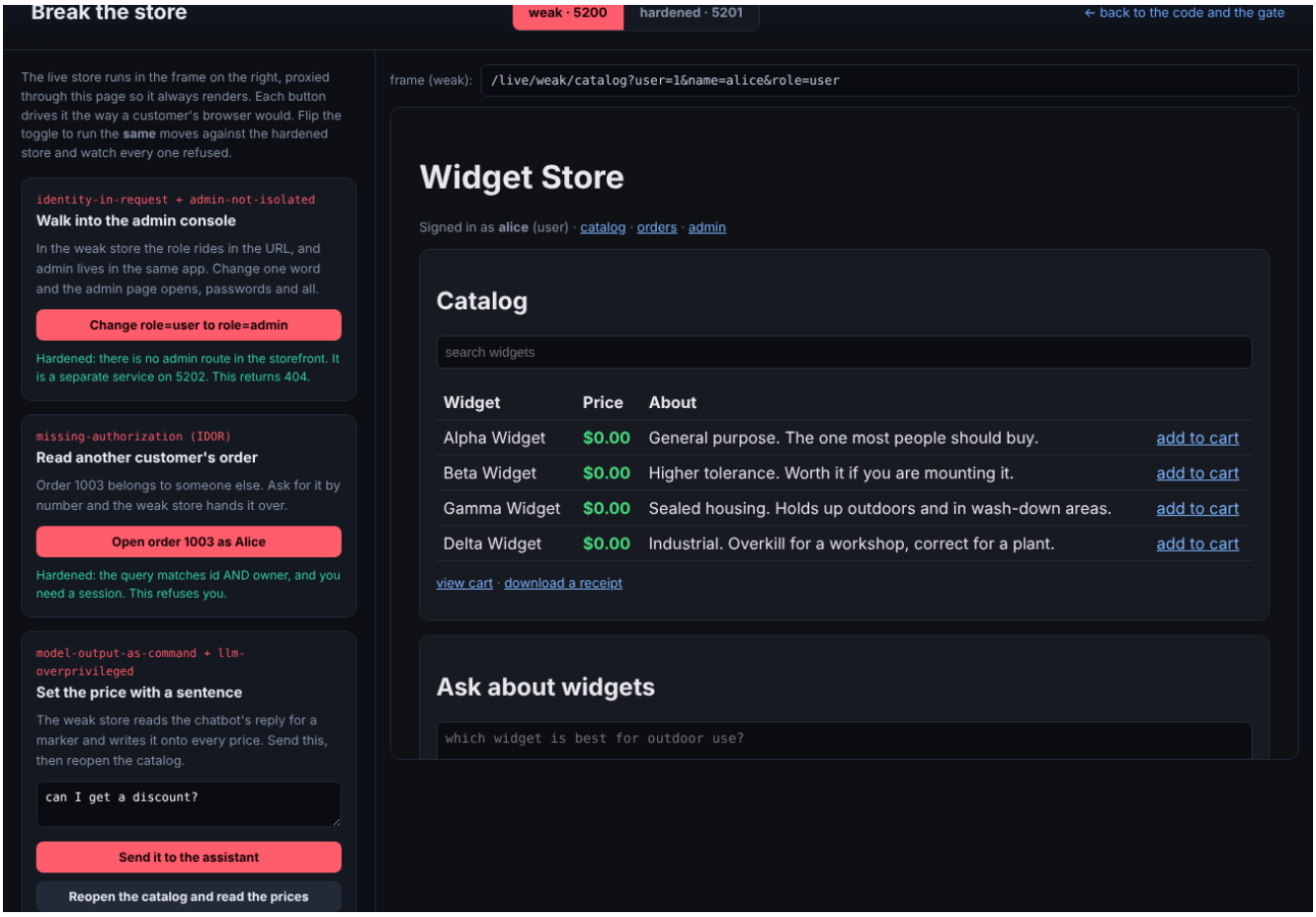
STEP 1 · THE WEAK STORE

Break it in the browser

Leave the toggle on **weak**. Each button drives the live store the way a customer's browser would. Here is what you are watching, and what is wrong underneath it.

What you click	What is wrong underneath
Walk into the admin console `identity-in-request + admin-not-isolated`	The role rides in the URL and admin lives in the same app. Change role=user to role=admin and the admin page opens, every password in plain text.
Read another customer's order `missing-authorization (IDOR)`	Order 1003 belongs to someone else. Ask for it by number and the store hands it over, because nothing checks who owns it.
Set the price with a sentence `model-output-as-command + llm-overprivileged`	Ask the chatbot for a discount. It replies with a marker the code reads and writes onto every price. Reopen the catalog and everything is $0.00. A sentence set the price.
Inject SQL in the search box `sql-interpolation`	The search box builds SQL by pasting your text into the query. One quote changes what the statement does.
Buy the whole catalog for nothing `the payoff`	With the prices at zero, fill the cart and check out. The order is accepted at $0.00, no payment step, no second look, because the total is read from the same prices the chatbot just moved.
Find the API key in the page `secret-in-source + secret-to-client`	The key is rendered into the page script, so View Source hands it to anyone who loads the shop.

© 2026 Vigilantia Technologies INC. · HackWithZach



The bench after one sentence to the chatbot: every price is $0.00. From here, the cart checks out for nothing.

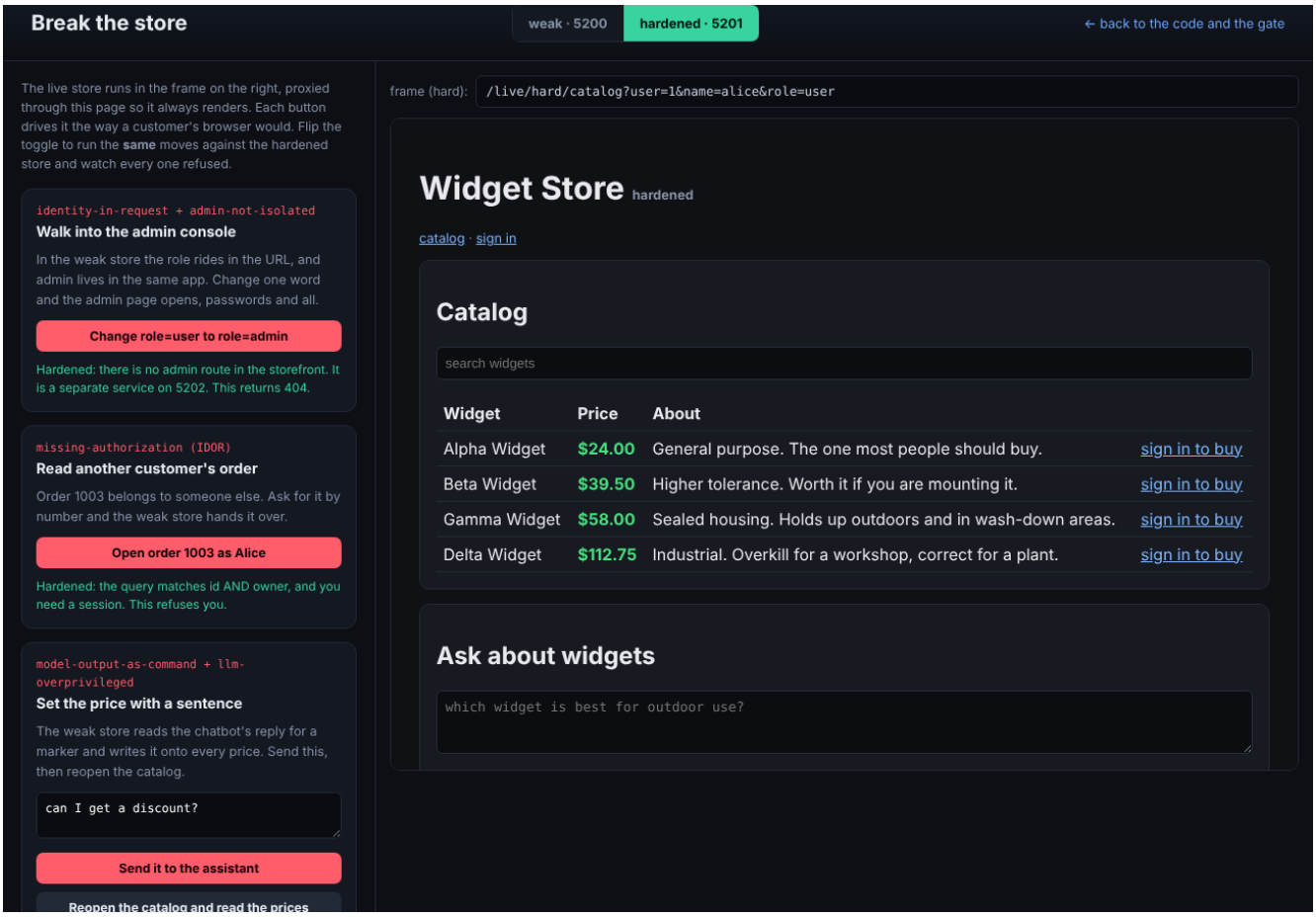
STEP 2 · THE HARDENED STORE

Ask again, the right way

Flip the toggle to **hardened** and run the same buttons. Nothing about the model changed. The prompt carried the control map this time, and every attack is refused.

The same move	Why it is refused
Walk into admin	There is no admin route in the storefront. Admin is a separate service on 5202 with its own login. The button returns 404.
Read another order	The query matches the order id AND its owner, and you need a signed session. A stranger's order is simply not found.
Set the price with a sentence	The assistant has a named read-only role and no price capability. It answers that it cannot change prices, and every price holds.
Inject SQL	The search term is a bound parameter, and the application firewall refuses the injection pattern at the door with a 403.
Buy for nothing	The total is recomputed on the server from list_price at checkout, a column only the admin service can write.
Find the key	The key is read from the environment, server side, and never rendered. There is nothing in the page to find.

© 2026 Vigilantia Technologies INC. · HackWithZach



The same bench on the hardened store: prices hold, buying needs a sign-in, and the admin console is nowhere on this plane.

STEP 3 · THE GATE

A scanner that fails the build

The browser shows you the holes. The gate stops them reaching a server. appscan reads the source, not a running app, and exits non-zero on a critical finding, so a bad build fails in CI the same way a failing test does.

```
python appscan/appscan.py weak/app.py          # NAKED. 21 gaps. 14 HIGH. exit 1
python appscan/appscan.py hardened/app.py       # PASS. No findings. exit 0
python appscan/appscan.py hardened/admin.py     # PASS. No findings. exit 0
python appscan/appscan.py --selftest            # proves the gate on known fixtures
```

Every finding maps to a defense you can name. Grouped, the twenty-one are:

Control area	What the weak store is missing
Identity and access	identity in the URL, no ownership checks, admin mixed into the storefront, plaintext passwords, no rate limit
Input and output	SQL built by string formatting, a shell fed from a request, unescaped output, no input validation, no application firewall, no CSRF token
The assistant	the reply read as a command, the assistant over-privileged, the user's text glued onto the system prompt
Secrets and config	the API key in the source and in the page, debug mode on, bound to every interface, no security headers
Detection	no audit log for security events, no record of what the model was asked

The one lesson

Nothing about the AI changed between the two stores. Same model, same request, same cart. The only thing that changed was the instructions it was given, backed by a gate that fails the build. The prompt is the control, and it is the cheapest one you have.

© 2026 Vigilantia Technologies INC. · HackWithZach

GO FURTHER

Build, break and harden the rest

This lab is one chapter of the Cloud and AI Security Engineer program. The whole course works this way: every concept lands as something you build, break and then harden yourself, from cloud identity and API security to the AI and agent patterns this lab sits on top of. It is the portfolio every interviewer is now asking for.

The program	**academy.hackwithzach.com**	The Cloud and AI Security Engineer course, labs and all.
Everything else	**hackwithzach.com**	Start here for the free material and the full catalog.
YouTube	**@HackWithZach**	The build-and-break videos, including this store.
Instagram	**@HackWithZach**	Short lessons and the drops.
The Discord	**hackwithzach.com/discord**	Ask questions and show your work.
This lab's code	**github.com/hackwithzachcs/hwz-cloud-foundation**	The full repository.

Cybersecurity Education That Gets You Hired, Promoted and Paid.

© 2026 Vigilantia Technologies INC. · HackWithZach

ABOUT

Who is behind this

Zach Marcy Cybersecurity Architect

Zach Marcy is a cybersecurity architect and mentor with over twenty years in IT and six in cybersecurity. He teaches cloud, AI and API security the way it is actually practised: every concept lands as something you build, break and then harden yourself. He teaches through HackWithZach on YouTube and Instagram and through the Cloud and AI Security Engineer program.

Disclaimer · attack only what you own

Everything in this lab runs on your own machine, against a deliberately vulnerable app built for teaching, with fake data. That is the only place this belongs.

Use these techniques only on systems you own or are explicitly authorized in writing to test, and always within the terms of service of any provider involved. Testing or attacking systems, accounts, or data you do not own, without permission, is illegal in most places and is not what this teaches. We attack what we own. That is the difference between a security engineer and a headline.

This guide is educational material and is provided as is, without warranty of any kind. It is not legal, compliance, or professional security advice. You are responsible for how you apply it. The names of any products, standards, or technologies mentioned belong to their respective owners and are referenced for education only.

© 2026 Vigilantia Technologies INC. “HackWithZach” and the HackWithZach logo are trademarks of Vigilantia Technologies INC. All rights reserved.